Relatório Final – Sistema de Fichas de Treino para Academia

Relatório final de projeto de software

Introdução

Este relatório apresenta o projeto de um sistema para gerenciamento de treinos de academia. O documento descreve o propósito do sistema, suas principais funcionalidades, as ferramentas e tecnologias utilizadas e os diagramas elaborados: o modelo conceitual, o modelo lógico (diagrama entidade-relacionamento) e o diagrama de classes.

Propósito do sistema

O sistema tem como objetivo permitir que o praticante de musculação organize seus treinos de forma personalizada e acompanhe sua evolução física. Em vez de depender de papéis ou anotações dispersas, o usuário passa a ter um ambiente único onde pode planejar seus exercícios e registrar suas medidas corporais ao longo do tempo.

As funcionalidades centrais são:

- **Gerenciamento de usuários:** cadastro e login.
- **Criação de fichas de treino:** o usuário pode criar quantas fichas desejar (por exemplo, "Treino A – Peito e Tríceps") e organizá-las da maneira que preferir.
- **Montagem de exercícios dentro das fichas:** cada ficha reúne um conjunto de exercícios, com informações como nome, carga, número de séries e repetições.
- **Registro de medidas corporais:** o usuário guarda peso, altura e a data de cada medição.

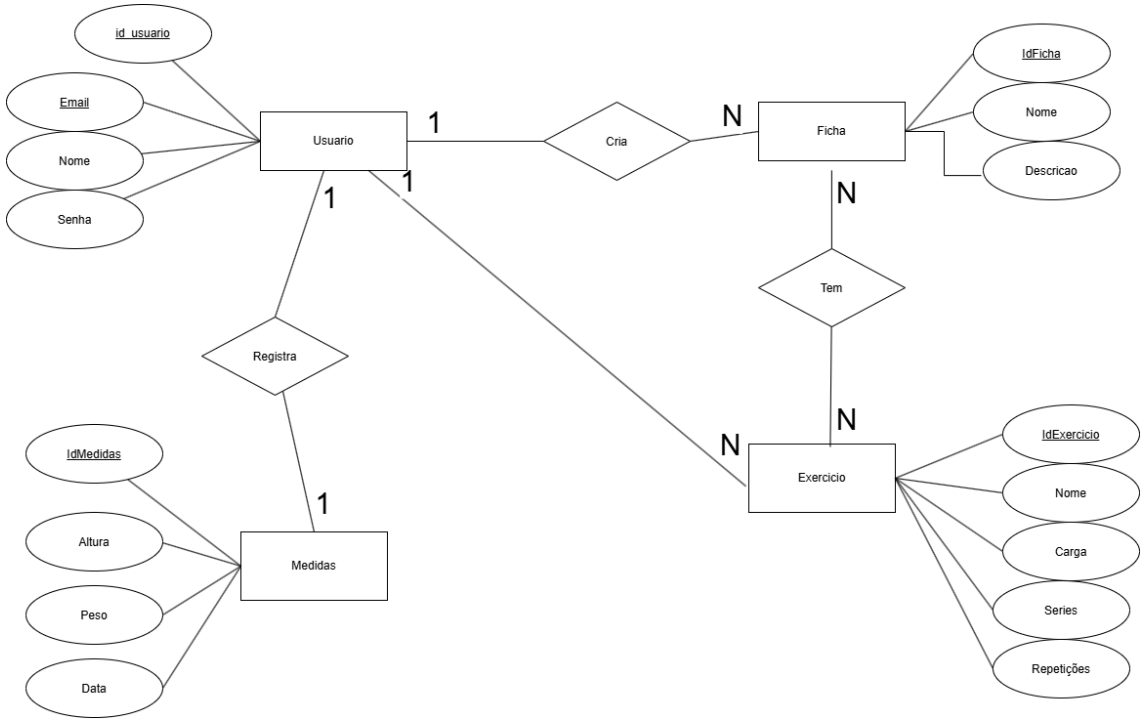
Modelo conceitual

O modelo conceitual representa as entidades do sistema e seus relacionamentos, sem considerar detalhes de implementação. Foram identificadas quatro entidades:

- **Usuário:** atributos Email (identificador), Nome e Senha.
- **Ficha:** atributos IdFicha (identificador) e Nome.
- **Exercício:** atributos IdExercício (identificador), Nome, Carga, Séries e Repetições.
- **Medidas:** atributos IdMedidas (identificador), Altura, Peso e Data.

Os relacionamentos definidos são:

Relacionamento	Entidades	Cardinalidade	Descrição
Cria	Usuário – Ficha	1:N	Um usuário cria várias fichas; cada ficha pertence a um único usuário.
Tem	Ficha – Exercício	N:N	Uma ficha contém vários exercícios, e um exercício pode aparecer em várias fichas.
Registra	Usuário – Medidas	1:1	Conforme representado no diagrama, cada usuário está associado a um registro de medidas.



Modelo lógico (Diagrama Entidade-Relacionamento)

O modelo lógico traduz o modelo conceitual para uma estrutura de tabelas relacionais, definindo chaves primárias (PK) e estrangeiras (FK). As tabelas são:

- **Usuario:** `id_usuario` (PK), `nome`, `senha`, `email`.
- **Ficha:** `id_ficha` (PK), `nome`, `descricao`, `id_usuario` (FK).
- **Exercicio:** `id_exercicio` (PK), `nome`, `grupo_muscular`, `carga`, `series`, `repeticoes`, `id_usuario` (FK).
- **Ficha_Exercicio:** `id_ficha_exercicio` (PK), `id_ficha` (FK), `id_exercicio` (FK).
- **Medidas:** `id_medida` (PK), `id_usuario` (FK, UNIQUE), `altura`, `peso`, `data`.

A relação muitos-para-muitos entre Ficha e Exercício foi resolvida com a tabela associativa **Ficha_Exercicio**, que é a abordagem padrão em bancos relacionais. A relação de Usuário com Ficha é de um para muitos, e a restrição UNIQUE em `id_usuario` na tabela Medidas implementa a relação um-para-um.

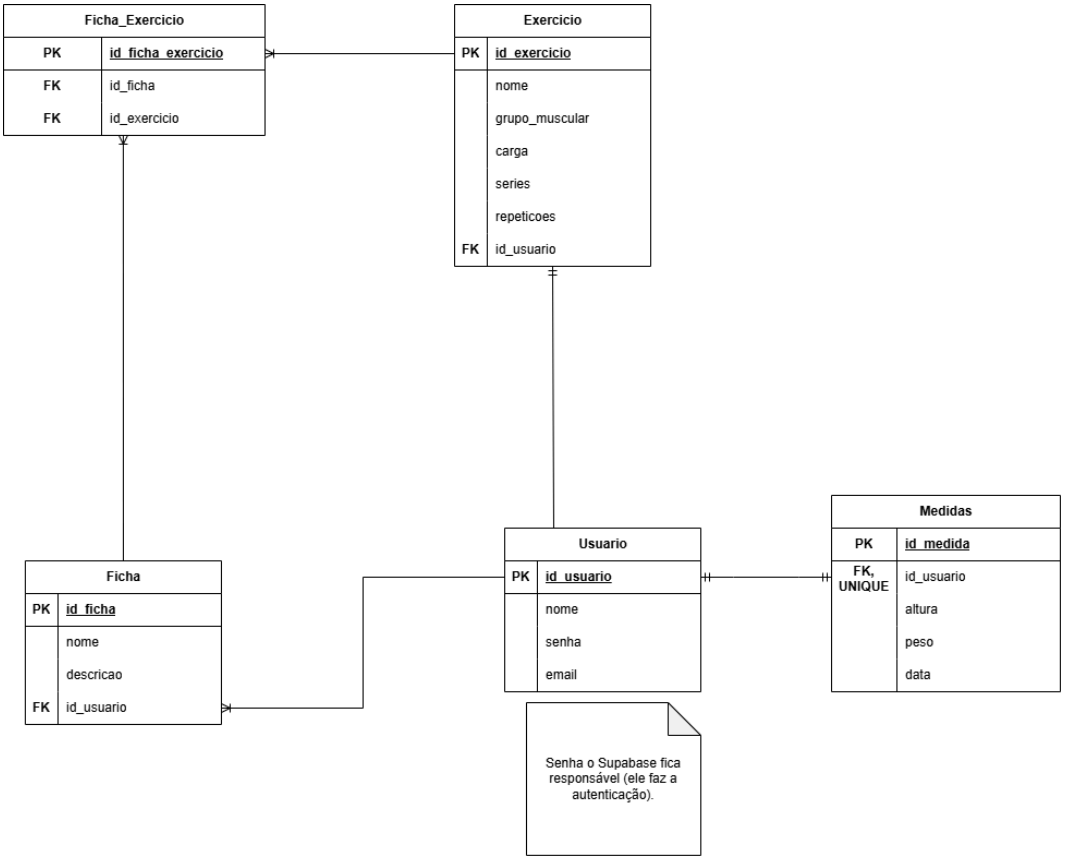
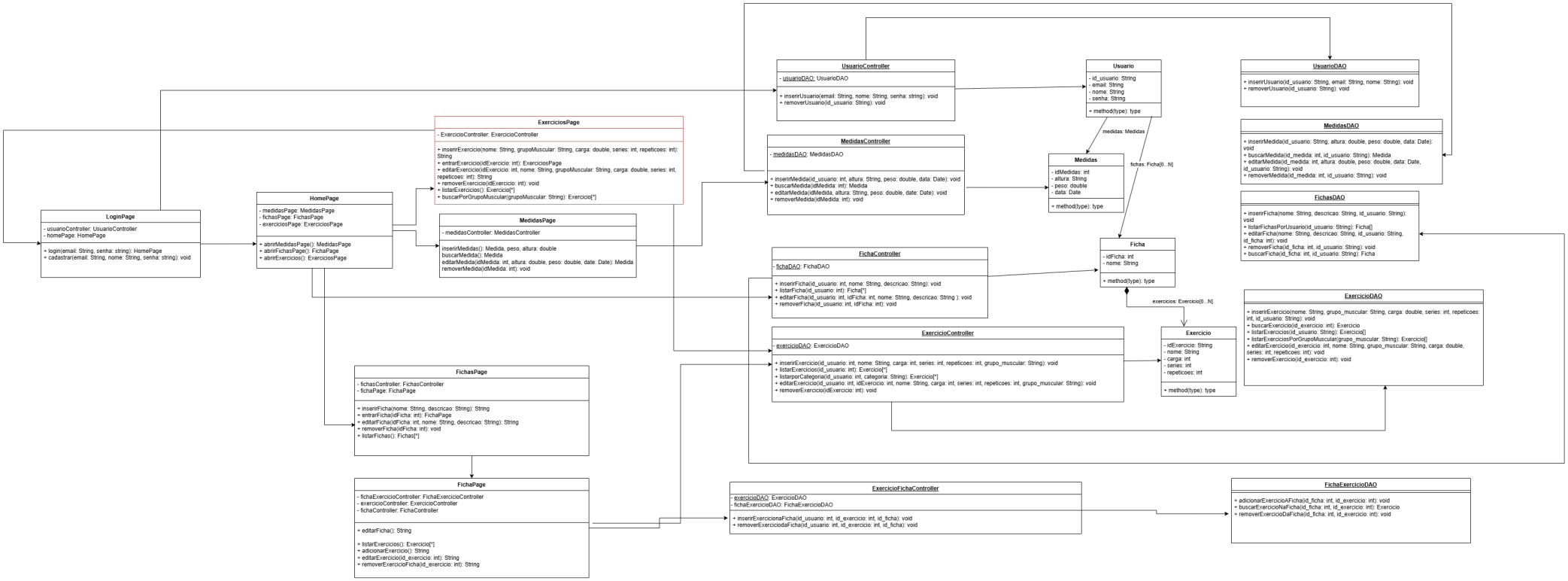


Diagrama de classes

O diagrama de classes descreve a arquitetura do software, organizada em camadas com responsabilidades distintas:

- **Camada de visão (Pages):** `LoginPage`, `HomePage`, `UserPage`, `MedidasPage` e `FichaPage`. Representam as telas do sistema e a navegação entre elas. A `HomePage` funciona como ponto central, abrindo as demais telas.
- **Camada de controle (Controllers):** `UsuarioController`, `MedidasController`, `FichaController` e `ExercicioController`. Recebem as ações das telas e coordenam as operações. Nos controllers foi feita a serialização.
- **Camada de modelo (entidades):** `Usuario`, `Medidas`, `Ficha` e `Exercicio`. Um usuário possui medidas e uma lista de fichas, e cada ficha é composta por uma lista de exercícios (relação de composição).
- **Camada de acesso a dados (DAOs):** `UsuarioDAO`, `MedidasDAO`, `FichasDAO` e `ExercicioDAO`. Concentram as operações de inserção, busca, edição e remoção no banco de dados. Nos DAOs foi feita a desserialização.

Essa organização separa interface, lógica e persistência, o que facilita a manutenção e futuras evoluções do sistema. Todas as entidades seguem o conjunto de operações CRUD (criar, ler, atualizar e remover).


LoginPage
- usuarioController: UsuarioController
- homePage: HomePage
+ login(email: String, senha: string): HomePage
+ cadastrar(email: String, nome: String, senha: string): void
HomePage
- medidasPage: MedidasPage
- fichasPage: FichasPage
- exerciciosPage: ExerciciosPage
+ abrirMedidasPage(): MedidasPage
+ abrirFichasPage(): FichaPage
+ abrirExercicios(): ExerciciosPage
ExerciciosPage
- ExercicioController: ExercicioController
+ inserirExercicio(nome: String, grupoMuscular: String, carga: double, series: int, repeticoes: int): String
+ entrarExercicio(idExercicio: int): ExerciciosPage
+ editarExercicio(idExercicio: int, nome: String, grupoMuscular: String, carga: double, series: int, repeticoes: int): String
+ removerExercicio(idExercicio: int): void
+ listarExercicios(): Exercicio[*]
+ buscarPorGrupoMuscular(grupoMuscular: String): Exercicio[*]
MedidasPage
- medidasController: MedidasController
inserirMedidas(): Medida, peso, altura: double
buscarMedida(): Medida
editarMedida(idMedida: int, altura: double, peso: double, date: Date): Medida
removerMedida(idMedida: int): void
UsuarioController
- usuarioDAO: UsuarioDAO
+ inserirUsuario(email: String, nome: String, senha: string): void
+ removerUsuario(id_usuario: String): void
Usuario
- id_usuario: String
- email: String
- nome: String
- senha: String
+ method(type): type
UsuarioDAO
+ inserirUsuario(id_usuario: String, email: String, nome: String): void
+ removerUsuario(id_usuario: String): void
MedidasController
- medidasDAO: MedidasDAO
+ inserirMedida(id_usuario: int, altura: String, peso: double, data: Date): void
+ buscarMedida(idMedida: int): Medida
+ editarMedida(idMedida, altura: String, peso: double, date: Date): void
+ removerMedida(idMedida: int): void
medidas: Medidas
Medidas
- idMedidas: int
- altura: String
- peso: double
- data: Date
+ method(type): type
MedidasDAO
+ inserirMedida(id_usuario: String, altura: double, peso: double, data: Date): void
+ buscarMedida(id_medida: int, id_usuario: String): Medida
+ editarMedida(id_medida: int, altura: double, peso: double, data: Date, id_usuario: String): void
+ removerMedida(id_medida: int, id_usuario: String): void
fichas: Ficha[0...N]
FichasDAO
+ inserirFicha(nome: String, descricao: String, id_usuario: String): void
+ listarFichasPorUsuario(id_usuario: String): Ficha[]
+ editarFicha(nome: String, descricao: String, id_usuario: String, id_ficha: int): void
+ removerFicha(id_ficha: int, id_usuario: String): void
+ buscarFicha(id_ficha: int, id_usuario: String): Ficha
FichaController
- fichaDAO: FichaDAO
+ inserirFicha(id_usuario: int, nome: String, descricao: String): void
+ listarFicha(id_usuario: int): Ficha[*]
+ editarFicha(id_usuario: int, idFicha: int, nome: String, descricao: String): void
+ removerFicha(id_usuario: int, idFicha: int): void
Ficha
- idFicha: int
- nome: String
+ method(type): type
exercicios: Exercicio[0...N]
ExercicioDAO
+ inserirExercicio(nome: String, grupo_muscular: String, carga: double, series: int, repeticoes: int, id_usuario: String): void
+ buscarExercicio(id_exercicio: int): Exercicio
+ listarExercicios(id_usuario: String): Exercicio[]
+ listarExerciciosPorGrupoMuscular(grupo_muscular: String): Exercicio[]
+ editarExercicio(id_exercicio: int, nome: String, grupo_muscular: String, carga: double, series: int, repeticoes: int): void
+ removerExercicio(id_exercicio: int): void
ExercicioController
- exercicioDAO: ExercicioDAO
+ inserirExercicio(id_usuario: int, nome: String, carga: int, series: int, repeticoes: int, grupo_muscular: String): void
+ listarExercicios(id_usuario: int): Exercicio[*]
+ listarporCategoria(id_usuario: int, categoria: String): Exercicio[*]
+ editarExercicio(id_usuario: int, idExercicio: int, nome: String, carga: int, series: int, repeticoes: int, grupo_muscular: String): void
+ removerExercicio(idExercicio: int): void
Exercicio
- idExercicio: String
- nome: String
- carga: int
- series: int
- repeticoes: int
+ method(type): type
FichasPage
- fichasController: FichasController
- fichaPage: FichaPage
+ inserirFicha(nome: String, descricao: String): String
+ entrarFicha(idFicha: int): FichaPage
+ editarFicha(idFicha: int, nome: String, descricao: String): String
+ removerFicha(idFicha: int): void
+ listarFichas(): Fichas[*]
FichaPage
- fichaExercicioController: FichaExercicioController
- exercicioController: ExercicioController
- fichaController: FichaController
+ editarFicha(): String
+ listarExercicios(): Exercicio[*]
+ adicionarExercicio(): String
+ editarExercicio(id_exercicio: int): String
+ removerExercicioFicha(id_exercicio: int): String
ExercicioFichaController
- exercicioDAO: ExercicioDAO
- fichaExercicioDAO: FichaExercicioDAO
+ inserirExercicionaFicha(id_usuario: int, id_exercicio: int, id_ficha): void
+ removerExerciciodaFicha(id_usuario: int, id_exercicio: int, id_ficha): void
FichaExercicioDAO
+ adicionarExercicioAFicha(id_ficha: int, id_exercicio: int): void
+ buscarExercicioNaFicha(id_ficha: int, id_exercicio: int): Exercicio
+ removerExercicioDaFicha(id_ficha: int, id_exercicio: int): void

Ferramentas e Tecnologias Utilizadas

Para o desenvolvimento do sistema foram utilizadas as seguintes ferramentas e tecnologias:

- **Banco de dados:** PostgreSQL.
- **Linguagem de programação:** JavaScript.
- **Controle de rotas:** Express.
- **Frontend:** Python, executado pelo terminal por enquanto.
- **Autenticação e servidor do banco de dados:** Supabase.
- **Testes das rotas:** Postman.
- **Versionamento:** Git e GitHub.

Para a colaboração em tempo real, foi utilizado o Live Share, extensão do VS Code usada para programar em conjunto durante as reuniões.

Quanto ao processo de desenvolvimento, foram realizadas reuniões diárias durante a semana passada e esta semana para desenvolver o código-fonte.